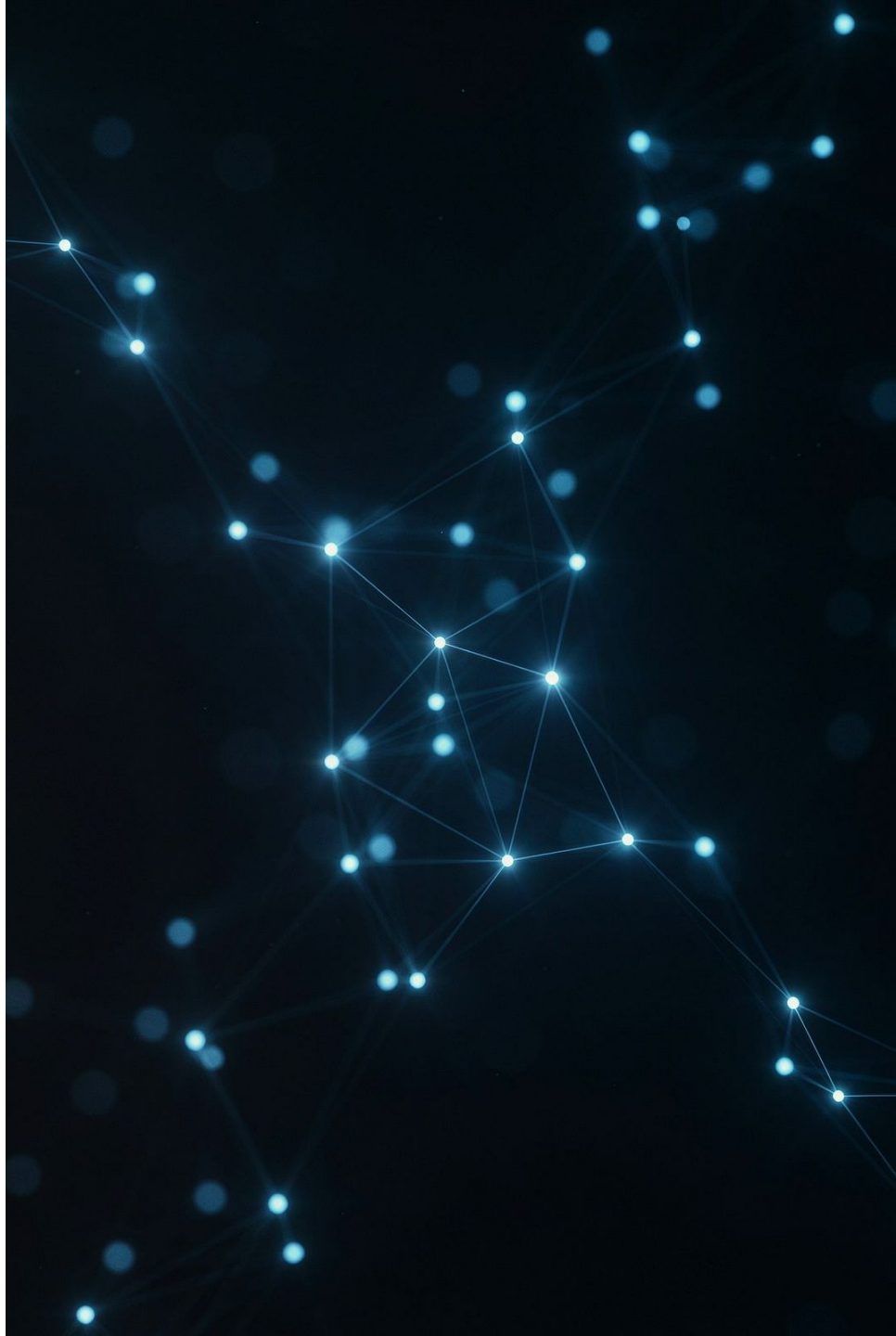
Sistemas de recomendación

SPRINT 9 · APRENDIZAJE NO SUPERVISADO Y POR REFUERZO · S09.2

TurisData vende 25 experiencias con los mismos banners. ¿Y si cada huésped viera las que más le pueden gustar?



Probar un recomendador: esconder notas

> Para saber si un recomendador es bueno hay que esconderle algunas valoraciones y ver si las adivina.

- Matriz usuario × experiencia (300 usuarios, 25 experiencias): las casillas vacías son NaN.
- Se reserva un 20 % al azar como prueba; la matriz se construye solo con el 80 % de entrenamiento.
- El usuario no ve notas, ve una lista corta: se mide con precision@k = de las k recomendadas, ¿qué fracción le gustó (≥ 4 estrellas en la prueba)? Solo se recomienda entre lo no valorado.

Tres enfoques

Enfoque	Idea	Pega
Popularidad	Lo mejor valorado para todos (media_item, Ejercicio 1)	No personaliza; es la referencia a batir y la única opción en arranque en frío
Contenido	Si te gusta el mar, más de mar (perfil por categoría, Ejercicio 3)	Se queda en lo que ya conoces
Colaborativo	Usuarios (o experiencias) parecidos, por similitud	Necesita historial y falla con usuarios nuevos

Ejercicio 2: precision_k; Ejercicio 4: experiencias parecidas a la 1.ª ordenadas por similitud.

Perfil por contenido, en una línea

```
# Ejercicio 3: suma de residuos por categoría / (nº de valoraciones + 3)
perfil_usuario = (C @ F) / ((M @ F) + 3)       # 300 × 7
```

- F: qué categoría tiene cada experiencia; C: residuos de cada usuario (nota – su media); M: qué ha valorado.
- El «+3» es prudencia: con pocas valoraciones en una categoría no nos fiamos.

Factorización de matrices: gustos ocultos

> Cada usuario tiene unos pocos gustos ocultos («aventura», «tranquilidad», «cultura») y cada experiencia una intensidad en cada uno. La nota esperada sale de multiplicarlos.

nota ≈ media + sesgo del usuario + sesgo de la experiencia + gustos · intensidades

```
P_mf = factorizar(R, k=3)                  # Ejercicio 5
rmse_mf, p_mf = evaluar_rmse(P_mf), precision_k(P_mf, 3)
mejor_rmse = min(metodos, key=lambda m: evaluar_rmse(metodos[m]))
```

Compara con RMSE (error de la nota) y con precision@3 (calidad de la lista).

Mini-reto: el recomendador de TurisData

```
def recomendar(id_usuario, n=3, P=P_mf):
    """n experiencias aún no valoradas con mayor nota prevista."""
    # columnas: nombre, categoria, isla, nota_prevista
```

- Copia P[id_usuario-1], pon -inf donde ya valoró, ordena y busca en experiencias.
- Prueba con los usuarios 1 y 2 y comprueba que sus listas son distintas.
- Parte del paso 4 del reto, junto con la nota de IA responsable: ¿qué pasa si solo recomendamos lo popular?

Ahora, a practicar

Lo que viene a continuación está en tu repositorio.

CUADERNO · Recomendadores: popularidad, contenido, colaborativo y factorización (sprints/S09/notebooks/S09_03_recomendadores)	60 min